Zelig Styler SDK — iOS Integration Guide

This guide covers embedding the Zelig Styler widget into a third-party iOS app. Shoppers build looks, try on outfits, and add items to cart; your app handles dismissal and cart operations through a small delegate / closure bridge.

The SDK ships as a **pre-built closed-source binary** (`ZeligStylerSDK.xcframework`) — no source code is included.

Prerequisites

- iOS 13.0+
- Xcode 14+
- Swift 5.7+
- Your **storeId** and **storeKey** — provided by your Zelig solutions engineer

Environments

The SDK binary is published per environment. **Integrators use staging and prod.**

> ***Current version: 1.0.0.*** *prod is pinned by version (`1.0.0`); staging follows its branch tip.*

Environment	Use	SPM Dependency Rule	Resolves to
prod	Production (shipping)	**Up to Next Major Version** from `1.0.0`	an immutable release
staging	Integration testing	**Branch** → `staging`	the latest staging build (rolling)

> *staging is the rolling testing channel; prod is the version-pinned production channel. The SDK behaves the same in both. Production apps always use prod.*

Step 1 — Install the SDK (SPM)

The SDK ships as a binary Swift package. The repo URL is the same for all environments — **which one you get is decided by the Dependency Rule, not the URL**.

1. Xcode: **File → Add Package Dependencies…**
2. Paste the git repo URL:

```
https://github.com/wearezelig/ZeligStylerSDK-binary.git
```

1. Set the **Dependency Rule** per the table above:
   - **Integration testing: Branch**, enter `staging`.
   - **Production: Up to Next Major Version**, starting at `1.0.0`.
2. **Add to Project** → select your app project → **Add Package**. In the sheet that appears, **Add to Target** → select your app target → **Add**.
3. Verify: the Project Navigator shows **zeligstylersdk-binary**, and your target's **General** → **Frameworks, Libraries, and Embedded Content** lists `ZeligStylerSDK`.

> *Both environments link the same module (`import ZeligStylerSDK`). To switch between staging and prod, change the package's **Dependency Rule** (Branch ↔ Version) under **Package Dependencies**, then **File → Packages → Update to Latest Package Versions**.*

Upgrading: Xcode → **File** → **Packages** → **Update to Latest Package Versions** (or **Reset Package Caches** if it won't refresh), rebuild, and ship. For staging, this also pulls the latest commit on the branch.

Step 2 — Import

```
import ZeligStylerSDK
```

If this compiles, the installation succeeded. If you see `Unable to resolve module 'ZeligStylerSDK'`: confirm the package dependency is added to your target (File → Packages → Reset Package Caches, then retry).

Step 3 — Store credentials

Your Zelig solutions engineer will provide your **storeId** and **storeKey**.

> ***Security:** treat `storeKey` like a password. Do not commit it to source control or embed it as a string literal in your binary. Load it from a secrets manager, a remote config endpoint, or your app's secure configuration layer.*

Code examples in this guide use a placeholder store; substitute your own credentials.

Step 4 — Basic integration

SwiftUI (recommended)

```
import SwiftUI
  import ZeligStylerSDK

  struct ProductDetailView: View {
      let productSku: String
      @State private var showStyler = false

      var body: some View {
          Button("Build a Look") { showStyler = true }
              .sheet(isPresented: $showStyler) {
                  ZeligStylerView(
                      config: ZeligStylerConfig(
                          storeId: AppConfig.zeligStoreId,
                          storeKey: AppConfig.zeligStoreKey,  // load from secure confi
  g — see Step 3
                          productCode: productSku,
                          userId: currentUserId   // logged-in user ID; nil for guests
                      ),
                      onAddToCart: { item, completion in
                          // Add the item to your cart, then resolve the widget's pendi
  ng state.
                          cart.add(sku: item.sku, size: item.size) { success in
                              completion(success)
                          }
                      },
                      onClose: { showStyler = false },
                      onError: { error in
                          print("Zelig error: (error)")
                      }
                  )
                  .ignoresSafeArea()
              }
      }
  }
```

UIKit

```
import UIKit
  import ZeligStylerSDK

  final class ProductViewController: UIViewController, ZeligStylerDelegate {
      private var styler: ZeligStyler?

      func showStyler(productCode: String) {
          let config = ZeligStylerConfig(
              storeId: AppConfig.zeligStoreId,
              storeKey: AppConfig.zeligStoreKey,  // load from secure config — see Step
  3
              productCode: productCode,
              userId: currentUserId   // logged-in user ID; nil for guests
          )
          let styler = ZeligStyler(config: config, delegate: self)
          self.styler = styler          // retain the reference
          styler.present(from: self)
      }

      func zeligStyler(_ styler: ZeligStyler,
                       didRequestAddToCart item: ZeligCartItem,
                       completion: @escaping (Bool) -> Void) {
          cart.add(sku: item.sku, size: item.size) { success in
              completion(success)
          }
      }

      // Optional — defaults to dismissing the styler.
      func zeligStylerDidRequestClose(_ styler: ZeligStyler) {
          styler.dismiss(animated: true)
      }

      // Optional — called on load or initialization failures.
      func zeligStyler(_ styler: ZeligStyler, didFailWith error: ZeligStylerError) {
          print("Zelig error: (error)")
      }
  }
```

Step 5 — The add-to-cart bridge

When a shopper taps **Add to Bag** in the widget, the SDK delivers a `ZeligCartItem` to your callback. **You must call `completion(true)` on success or `completion(false)` on failure** — the widget updates its in-modal UI from this signal. Respond promptly: the widget has a ~2–4 s fallback timeout, after which it resolves the pending state automatically. If that happens before your callback fires, the widget's displayed state may fall out of sync with your cart.

SwiftUI — the `onAddToCart` closure:

```
onAddToCart: { item, completion in
      cartService.add(sku: item.sku, size: item.size, quantity: item.quantity ?? 1) { r
  esult in
          switch result {
          case .success: completion(true)
          case .failure: completion(false)
          }
      }
  }
```

UIKit — the `ZeligStylerDelegate` method:

```
func zeligStyler(_ styler: ZeligStyler,
                   didRequestAddToCart item: ZeligCartItem,
                   completion: @escaping (Bool) -> Void) {
      cartService.add(sku: item.sku, size: item.size, quantity: item.quantity ?? 1) { r
  esult in
          switch result {
          case .success: completion(true)
          case .failure: completion(false)
          }
      }
  }
```

`ZeligCartItem` fields:

Field	Description
`sku`	Normalized product SKU.
`size`	Selected size string (`"all"` for one-size items).
`quantity`	Quantity if the widget supplied one; otherwise `nil`.
`price`	Convenience accessor for the widget's price field, if present.
`raw`	The full widget payload (`[String: Any]`) for any extra fields.

Step 6 — Configuration reference

Parameter	Type	Required	Default	Description
`storeId`	`String`	Yes	—	Your Zelig store identifier.
`storeKey`	`String`	Yes	—	Your store API secret, paired with `storeId`.
`productCode`	`String`	Yes for `.productDetail`	—	Hero SKU to open on. Pass `""` for `.buildALook` / `.myCloset`.
`userId`	`String?`	No	`nil`	Your app's logged-in user ID. See Step 7.
`entry`	`ZeligStylerEntry`	No	`.productDetail`	Which experience to open. See "Entry modes" below.

Entry modes

```
public enum ZeligStylerEntry { case productDetail, buildALook, myCloset }
```

- `.productDetail` *(default)* — open on a specific product. `productCode` must be non-empty.
- `.buildALook` — open the Build a Look experience with no host-provided product. The widget uses the shopper's last look or the store's default recommendation. Pass `productCode: ""`.
- `.myCloset` — same as `.buildALook`, but lands directly on the My Closet tab.

```
// Open on a specific product (e.g. from a PDP):
  ZeligStylerConfig(storeId: AppConfig.zeligStoreId, storeKey: AppConfig.zeligStoreKey,
  productCode: sku)

  // Entry points with no specific product:
  ZeligStylerConfig(storeId: AppConfig.zeligStoreId, storeKey: AppConfig.zeligStoreKey,
  productCode: "", entry: .buildALook)
  ZeligStylerConfig(storeId: AppConfig.zeligStoreId, storeKey: AppConfig.zeligStoreKey,
  productCode: "", userId: currentUserId, entry: .myCloset)
```

Step 7 — Shopper identity & closet

`userId` tells the Zelig backend how to scope the shopper's closet:

- **Non-nil (signed in)** — the closet is tied to this user and syncs across devices. An anonymous closet is automatically inherited on first sign-in.
- `nil` **(guest)** — a device-local anonymous closet is used instead.

```
let config = ZeligStylerConfig(
      storeId: AppConfig.zeligStoreId,
      storeKey: AppConfig.zeligStoreKey,  // load from secure config — see Step 3
      productCode: productCode,
      userId: currentUserId   // your app's real user ID; nil for guests
  )
```

Step 8 — Error handling

Implement the optional error callback to surface load or initialization failures:

```
// SwiftUI — onError closure (see Step 4)
  // UIKit — ZeligStylerDelegate method:
  func zeligStyler(_ styler: ZeligStyler, didFailWith error: ZeligStylerError) {
      // Possible values:
      // .loaderLoadFailed(URL)
      // .widgetInitTimeout
      // .modalMountTimeout
      // .webViewNavigationFailed(underlying: Error)
      // .hostPageUnavailable
  }
```

Your responsibilities:

1. Always call the add-to-cart `completion` — on both success and failure.
2. Ensure the device has network access before presenting the styler.

Step 9 — Testing & debugging

Use the SKUs provided by your Zelig solutions engineer for your store.

In debug builds, connect **Safari → Develop → (your device or simulator) → Zelig Styler** to inspect the embedded `WKWebView`. The SDK forwards JavaScript `console.log` / `console.error` output to the Xcode console — filter for `[Zelig iOS]` to isolate SDK messages.

Step 10 — SDK versions & updates

Widget updates (UI changes, algorithms, new features) ship from Zelig's CDN with no action required on your part. The widget bundle loads at runtime; shoppers receive the latest version the next time they open the styler.

You only need to update the SDK itself when Zelig releases a new native version (bridge or API changes — your Zelig solutions engineer will notify you in advance):

- Xcode → **File** → **Packages** → **Update to Latest Package Versions**, rebuild, and ship.

The SDK follows semantic versioning. Any `1.x` update is fully backward-compatible with your existing integration code.

Step 11 — Production checklist

- [ ] Production `storeId` / `storeKey` in use (not staging credentials).
- [ ] SPM Dependency Rule set to **Up to Next Major Version (1.0.0)** — confirmed not on the staging branch before shipping.
- [ ] `storeKey` not committed to source control.
- [ ] `productCode` passed and present in your store's catalog.
- [ ] Add-to-cart callback implemented; `completion` called on both success and failure.
- [ ] Logged-in `userId` passed for authenticated shoppers (`nil` only for guests).
- [ ] Tested on multiple device sizes (phone and tablet).

Troubleshooting

Symptom	Fix
`Missing package product 'ZeligStylerSDK'`	The package dependency isn't added to your target. File → Packages → Reset Package Caches; confirm your target has `ZeligStylerSDK` checked.
`Unable to resolve module 'ZeligStylerSDK'`	The Dependency Rule doesn't match the repo state (tag was moved / branch changed). Reset Package Caches then Update; or delete `Package.resolved` and re-resolve.
Widget blank or not loading	Check network connectivity; verify `storeId` / `storeKey`; open Safari Web Inspector and filter for `[Zelig iOS]` logs.
Add to cart hangs or never resolves	Confirm your `onAddToCart` / delegate method is wired up and that `completion(true/false)` is called on every code path.
Wrong product opens	Verify `productCode` is correct and exists in your store's catalog.
Touches not registering on a physical device	Contact your Zelig solutions engineer — this is typically a host-app layout configuration issue.

API reference

```
public enum ZeligStylerEntry { case productDetail, buildALook, myCloset }

  public struct ZeligStylerConfig {
      public let storeId: String
      public let storeKey: String
      public let productCode: String
      public let userId: String?
      public let entry: ZeligStylerEntry

      public init(
          storeId: String,
          storeKey: String,
          productCode: String,
          userId: String? = nil,
          entry: ZeligStylerEntry = .productDetail
      )
  }

  public struct ZeligCartItem {
      public let sku: String
      public let size: String
      public let quantity: Int?
      public let raw: [String: Any]
      public var price: String? { get }
  }

  public enum ZeligStylerError: Error {
      case loaderLoadFailed(URL)
      case widgetInitTimeout
      case modalMountTimeout
      case webViewNavigationFailed(underlying: Error)
      case hostPageUnavailable
  }

  public protocol ZeligStylerDelegate: AnyObject {
      // Required:
      func zeligStyler(_ styler: ZeligStyler,
                       didRequestAddToCart item: ZeligCartItem,
                       completion: @escaping (Bool) -> Void)
      // Optional:
```

```
    func zeligStylerDidRequestClose(_ styler: ZeligStyler)
    func zeligStyler(_ styler: ZeligStyler, didFailWith error: ZeligStylerError)
}

public final class ZeligStyler {
    public init(config: ZeligStylerConfig, delegate: ZeligStylerDelegate? = nil)
    public func present(from presenter: UIViewController, animated: Bool = true)
    public func dismiss(animated: Bool = true)
}

public struct ZeligStylerView: UIViewControllerRepresentable {
    public init(
        config: ZeligStylerConfig,
        onAddToCart: @escaping (ZeligCartItem, @escaping (Bool) -> Void) -> Void,
        onClose: (() -> Void)? = nil,
        onError: ((ZeligStylerError) -> Void)? = nil
    )
}
```

Support

Contact your Zelig solutions engineer for store credentials, environment configuration, and any merchant-specific setup.

Version: 1.0.0 · **Last updated:** July 2026